AGENTIC AI SYSTEMS · LECTURE 1

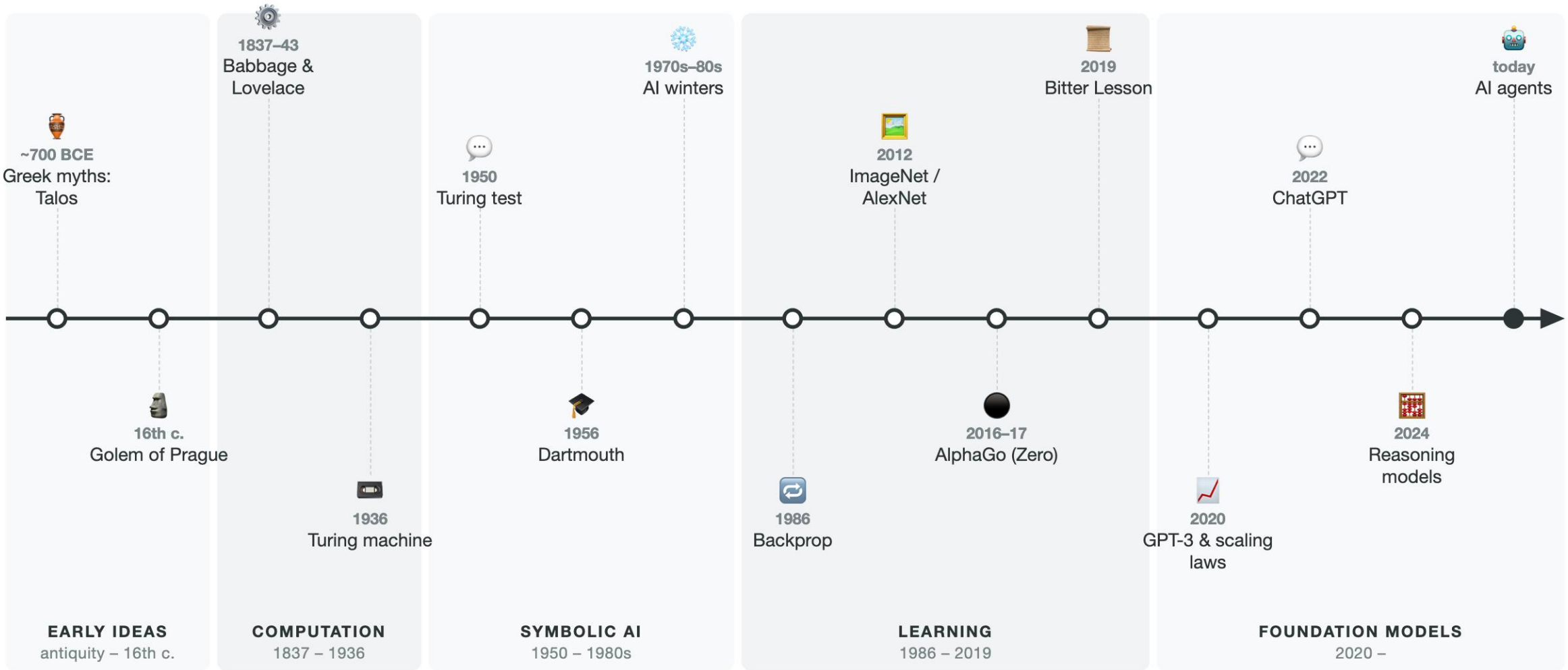
A Historical Overview of AI

Yonathan Efroni
Tel Aviv University

Artificial intelligence:

the hypothesis that intelligence is an engineering problem.

A Historical Overview of AI



PART I

Early ideas

Talos (טאלוס): the first robot?

- A **bronze automaton** built by Hephaestus (הפייסטוס) to guard Crete
- Destroyed through a single weak point — a bronze nail

Imagined for millennia — a science only recently.



Death of Talos, c. 400 BCE · Ruvo di Puglia (CC BY-SA 4.0)

Mayor, *Gods and Robots* (2018)

The Golem of Prague

- Clay being animated by the **Maharal** to carry out instructions
- Follows orders *literally* — and gets out of control
- The Maharal dismantles it

AI safety is an old question.

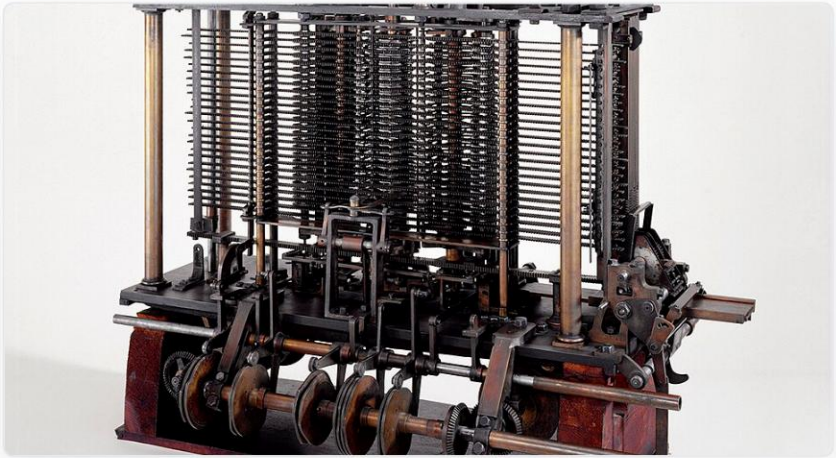


Mikoláš Aleš, 1899 (public domain)

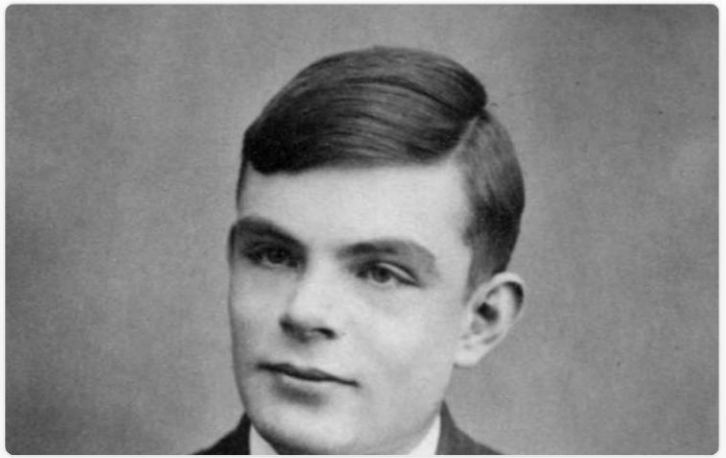
PART II

Computation

The universal machine







1837–1843

Babbage & Lovelace

Analytical Engine: a programmable *mechanical* machine

1936

Turing

Universal computation

One machine can run any procedure.

PART III

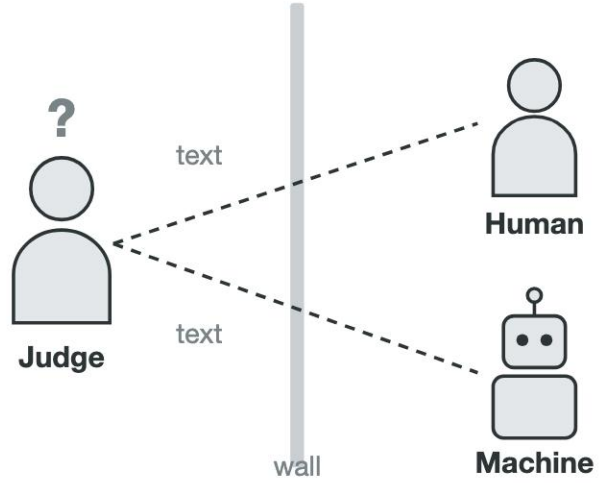
Symbolic AI

The Turing test (1950)



Alan Turing, 1951

- "Can machines think?" → an **observable test**: the *imitation game*
- A judge chats, by text only, with a hidden human and a hidden machine — can the judge tell which is which?



The first AI benchmark came before the first AI system.

Dartmouth (1956): AI becomes a field

> *"…every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it."*

Intelligence becomes an explicit engineering objective.



Back (left to right): Selfridge, Rochester, Minsky, McCarthy
Front: Solomonoff (left), Shannon (right)
Photo: courtesy of the Minsky family

Symbolic AI: program the rules

Intelligence as **rules + logic + search**. Main research directions:

- **Theorem proving** — derive proofs by applying logical rules (Logic Theorist, 1956)
- **Planning** — search for a sequence of actions that reaches a goal (GPS, STRIPS)
- **Game playing** — search over future moves (checkers, chess)
- **Expert systems** — encode a specialist's knowledge as if–then rules (MYCIN)

> **Bottleneck:** the real world needs too many rules to write by hand.

AI winters (1970s, late 1980s): promises outran results.

PART IV

Learning

Neural networks: learn the rules

Instead of designing rules by hand, **learn them from data**: pick a parameterized model and adjust its parameters to fit examples.

Perceptron (Rosenblatt, 1958)

$$w \leftarrow w + \eta\,(y - \hat{y})\,x, \qquad \hat{y} = \operatorname{sign}(w^\top x)$$

Update only on mistakes.

Backpropagation (Rumelhart, Hinton & Williams, 1986)

$$\theta \leftarrow \theta - \eta\,\nabla_\theta\,\mathcal{L}\big(f_\theta(x),\,y\big)$$

Gradient of a multi-layer network, via the chain rule.

Both are variants of (stochastic) gradient descent.

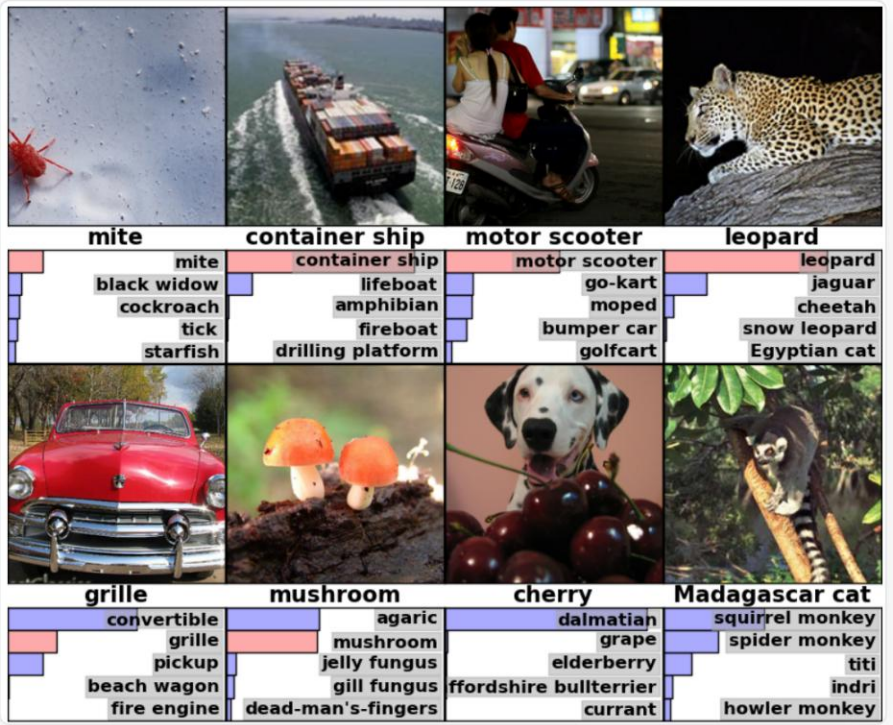
ImageNet & AlexNet (2012)

A deep neural network beat all alternatives by a wide margin: **15.3%** vs. **26.2%** top-5 error.

Three ingredients made it work:

- **Capability** — recognize objects in images
- **Measurement** — 1.2M labeled images + a standard test set
- **Optimization** — a deep CNN trained with backprop on GPUs

Trained on **cross-entropy**, evaluated on **accuracy**.

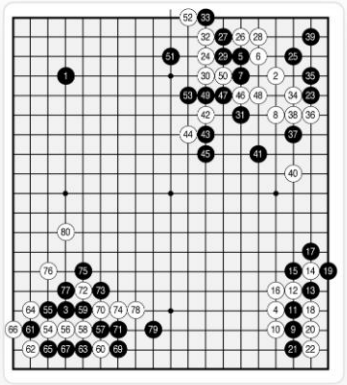


Test images and the model's top-5 predictions

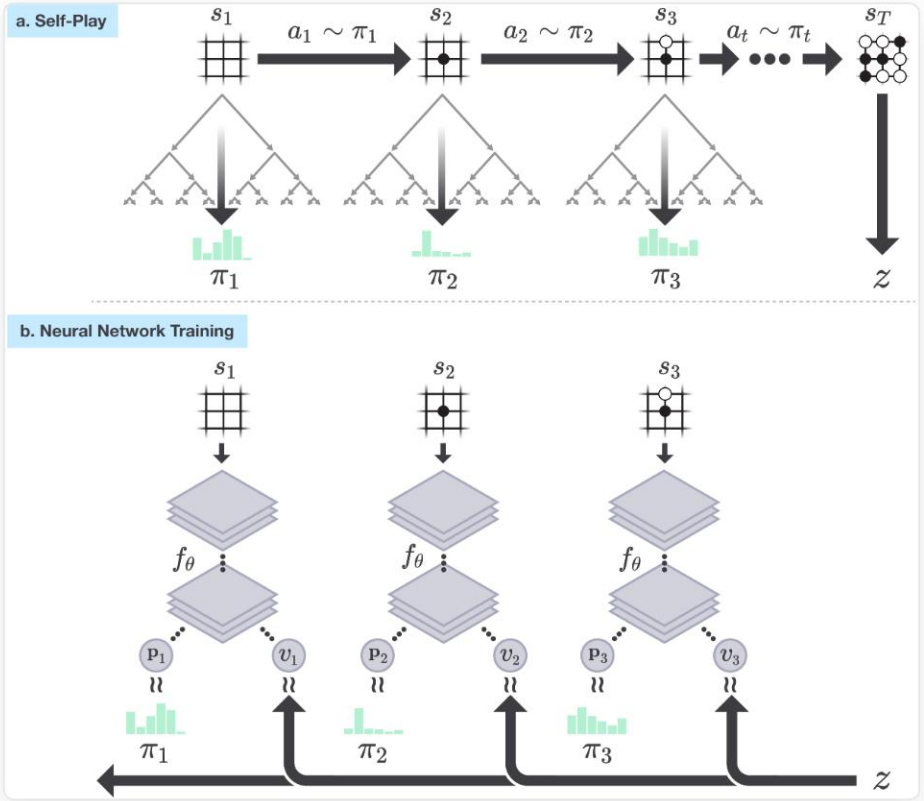
Krizhevsky, Sutskever & Hinton (2012), Fig. 4

AlphaGo Zero (2017)

- Learned Go **from scratch**, with no human games, using reinforcement learning.
- **Self-play:** tree search finds better moves; the network learns to predict them and the winner.

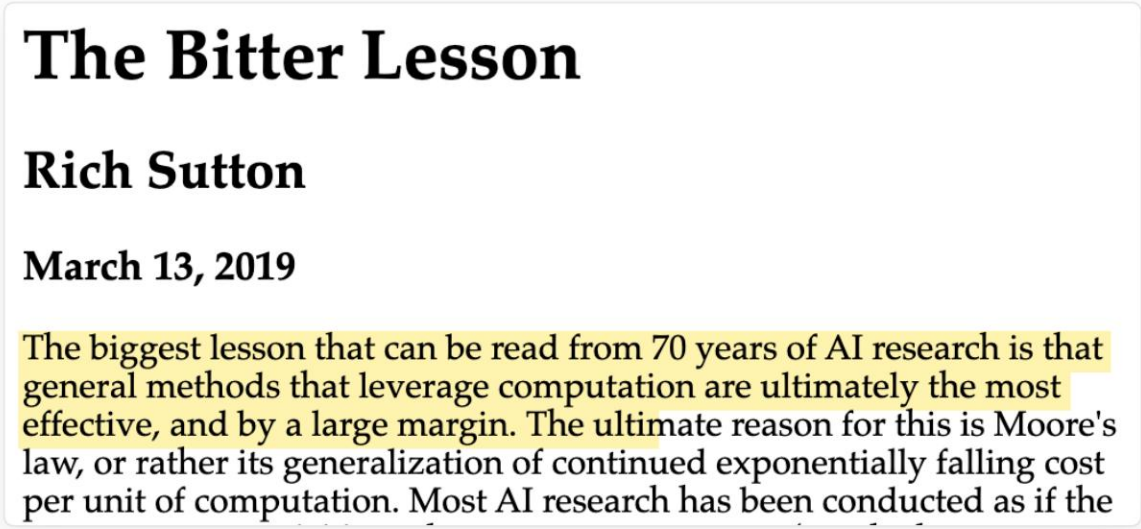


A self-play game after ~70 hours of training (Fig. 5c)



The Bitter Lesson (2019)

- 70 years of AI: **general methods that leverage computation** win, by a large margin.
- Hand-built domain knowledge helps in the short term, then plateaus.
- The two methods that scale with compute: **search** and **learning**.



The Bitter Lesson

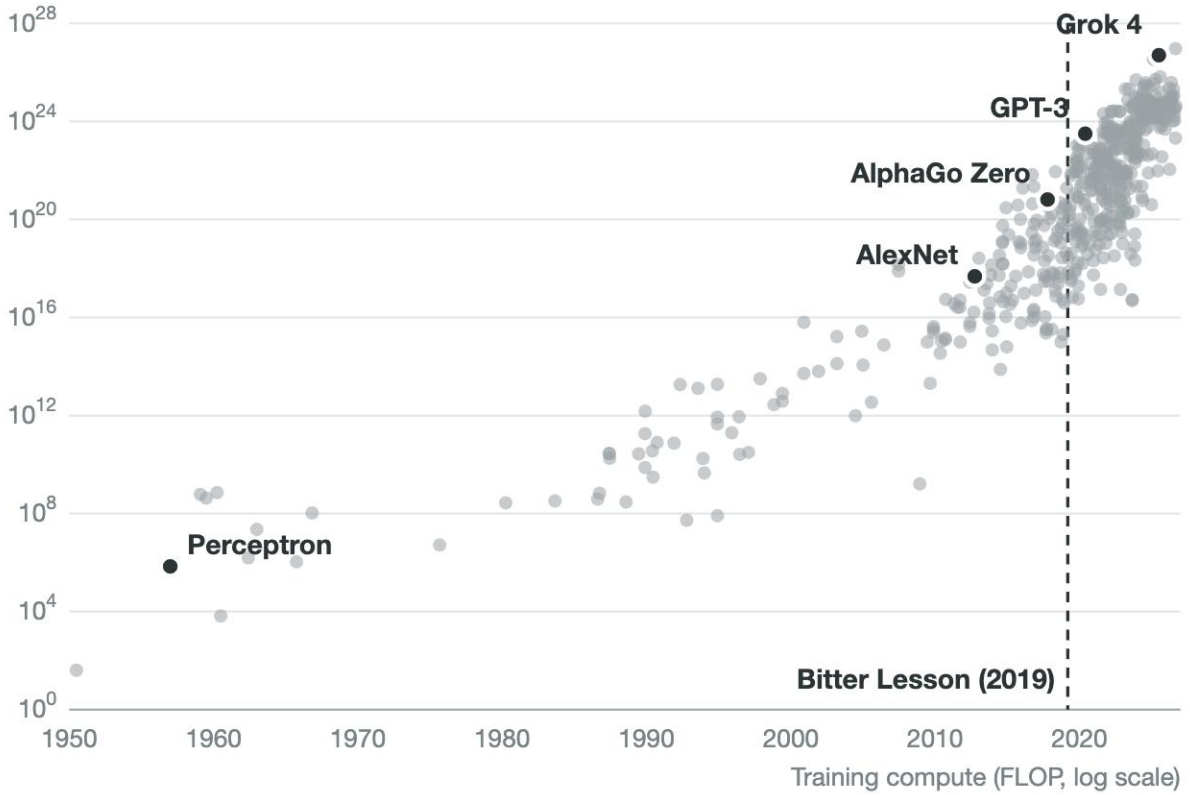
Rich Sutton

March 13, 2019

The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore's law, or rather its generalization of continued exponentially falling cost per unit of computation. Most AI research has been conducted as if the

Rich Sutton, *The Bitter Lesson*, March 13, 2019

The field bets on compute



- Training compute of notable models grows **~4× per year** since 2010.
- The essay named a trend — and it became the field's strategy.
- **Scaling laws** (2020) made the bet predictable: loss falls as a power law in compute.

Data: Epoch AI, *Notable AI Models* (CC BY) · 540 models with known compute · growth = log-linear fit, 2010–2026

PART V

Foundation models

Transformers & large-scale pretraining

Train one large model to **predict the next word** on a huge, diverse corpus of text.

- **Transformer** (2017): an architecture that trains efficiently at scale.
- Training objective — next-token prediction:

$$\mathcal{L}(\theta) = -\sum_{t} \log p_\theta\left(x_t \mid x_{<t}\right)$$

This is not the capability we care about — yet at scale the model can translate, answer questions and solve new tasks from a few examples in the prompt (GPT-3, 2020).



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

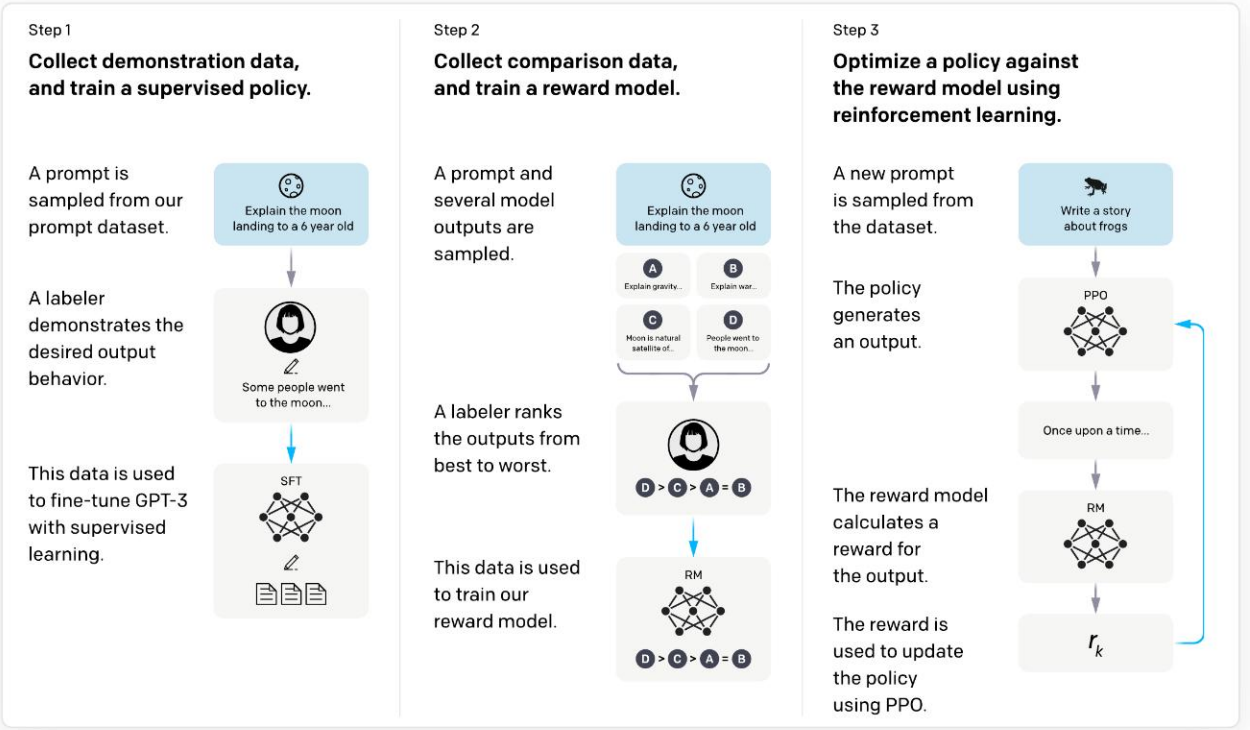
Vaswani et al., NIPS 2017

The GPT moment: one model, many tasks

One model, instructed in plain language, does many different tasks.

- Answering questions
- Writing
- Coding
- Reasoning
- Using tools

ChatGPT (2022), GPT-4 (2023)



Ouyang et al., *Training language models to follow instructions with human feedback* (2022)

Reasoning models (2024–)

Train models to **think step by step** before answering, on problems whose answers can be checked automatically.

The idea of training reasoning models:

- **Capability** — math & code reasoning
- **Measurement** — verifiable answers (checkers, unit tests)
- **Optimization** — reinforcement learning + thinking longer (inference-time compute)

The AlphaGo Zero recipe, applied to language models.

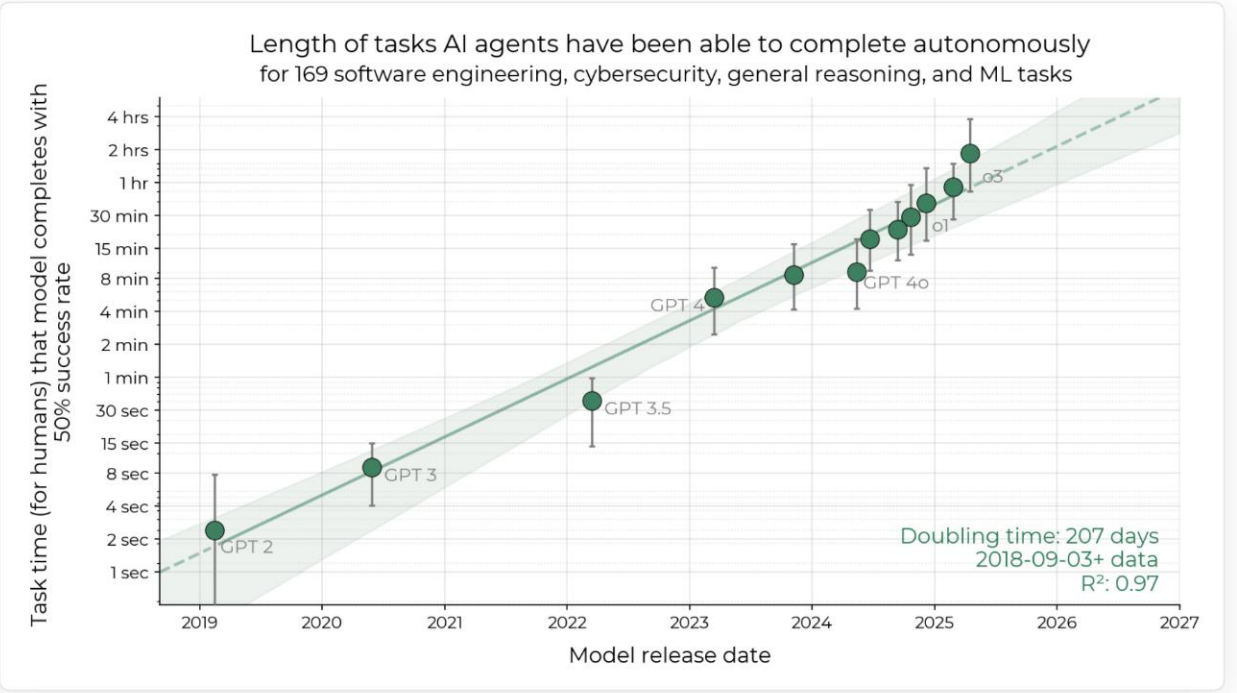
Today's frontier: Claude Opus · ChatGPT · Muse · Kimi · GLM · DeepSeek

The frontier

AI is moving from answering questions to **acting autonomously** over long tasks.

- **Prediction** (ImageNet)
- **Generation** (MMLU)
- **Reasoning** (Humanity's Last Exam)
- **Acting** (SWE-bench, OSWorld)
- **Long-horizon autonomy** (RE-Bench)

Harder capabilities need richer evaluation environments.



Kwa et al., *Measuring AI Ability to Complete Long Software Tasks*, METR (2025)

TAKEAWAYS

What have we learned?

A recurring pattern

Major breakthroughs combine three steps:

(1) Define a capability → (2) Develop scalable measurement → (3) Optimize at scale

	Capability	Feedback	Optimization
AlexNet	Image recognition	Labeled images	Supervised learning
AlphaGo Zero	Go	Game outcomes	RL + search
GPT	Broad language	Next token on diverse text	Self-supervised
Reasoning models	Math & code	Verifiable answers	RL + inference compute

An interpretation of history — not a law.

Step 1: define a capability

To engineer a capability, we must say what success looks like.

- **Abstract capability** — "software engineering ability"
- **Operational task** — "resolve this GitHub issue"
- Other examples: recognize an object, win a game, solve a math problem.

Step 2: scalable measurement

To optimize, we need feedback that scales.

- **Volume** — many examples.
- **Diversity** — cover the situations that matter.
- **Cost & reliability** — cheap, accurate, ideally automatic.

Harder to measure: Go → math → software → science.

Training feedback ≠ scientific evaluation — the topic of Lecture 2.

Step 3: optimize at scale

Improve performance by spending more compute.

- **Learning** — supervised, self-supervised, reinforcement learning.
- **Search** — explore many possibilities (game trees, planning).
- **Inference-time compute** — let the model think longer.

Scaling laws: loss falls predictably, as a power law, with compute, data and model size — $\mathcal{L}(C) \propto C^{-\alpha}$. (Kaplan et al., 2020)

General methods that scale with compute win — but only with data and evaluation that scale too.

In this course

1 **Benchmarks**

Datasets, evaluations, and environments.

2 **Harness**

What turns a model into an agent.

3 **AI safety**

How can we monitor autonomous agents?

4 **Post-training**

From pretrained model to assistant.